Deep Residual Networks: A Visual and Mathematical Exploration

Chandra Prakash Bathula

November 2025

Chapter 1

Introduction to Deep Residual Networks (ResNets)

1.1 From Shallow to Deep: The Road to Residual Learning

The journey of convolutional neural networks (CNNs) has been a story of going deeper. Early networks like **LeNet-5** (1998) contained only a few layers and were sufficient for digit recognition. A decade later, **AlexNet** (2012) demonstrated that depth, combined with GPUs and ReLU activations, could scale performance to the ImageNet challenge. **VGG-16/19** (2014) extended this idea—stacking 3×3 convolutions to reach 19 layers—but training deeper models soon hit a wall: beyond a certain depth, both training and test errors increased. This "**degradation problem**" puzzled researchers; deeper networks should, in principle, perform at least as well as their shallower counterparts.

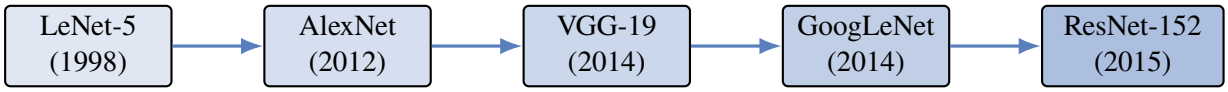


Figure 1.1: Milestones in CNN evolution: each leap involved greater depth and improved regularization until ResNets redefined optimization.

1.2 Why Going Deeper Fails Without Residuals

Vanishing Gradients

When many nonlinear layers are stacked, gradient magnitudes shrink exponentially as they back-propagate, effectively freezing early layers.

Degradation Problem

Even with BatchNorm and ReLU, deeper plain CNNs exhibit higher training error than shallower ones—evidence of optimization difficulty, not overfitting.

Optimization Barriers

The loss surface of very deep nets contains plateaus and saddle points. Without a direct signal path, optimization becomes inefficient and unstable.

1.3 Enter ResNets: The Identity Shortcut Revolution

Proposed by **Kaiming He et al.** in 2015, the **Deep Residual Network (ResNet)** introduced the idea of *residual learning*—each block explicitly learns the residual $\mathcal{F}(x) = \mathcal{H}(x) - x$ instead of the full mapping $\mathcal{H}(x)$:

$$\mathcal{H}(x) = x + \mathcal{F}(x).$$

If an identity mapping suffices, $\mathcal{F}(x)$ becomes zero, allowing the network to bypass redundant transformations automatically. This simple algebraic trick enabled stable training of networks exceeding 1000 layers and cut ImageNet top-5 error from 10.1

1.4 How Residual Learning Works Intuitively

- **Information Highway:** The shortcut path ensures signals and gradients travel unimpeded from early to late layers.
- **Ease of Optimization:** Instead of relearning a function, each block fine-tunes residual corrections.
- **Modularity:** Residual blocks can be stacked to arbitrary depth without architectural redesign.

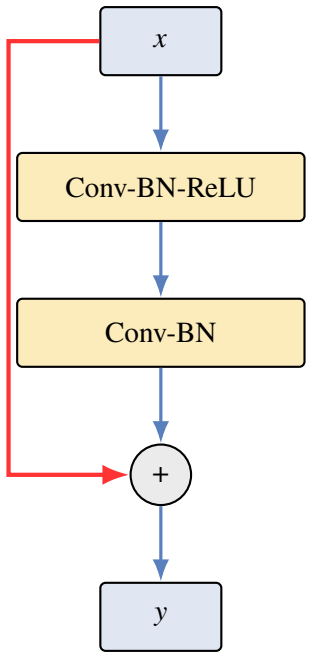


Figure 1.2: Core idea: a residual block learns only the difference between the desired mapping and the identity.

1.5 Historical Impact

Since their introduction, ResNets have become a universal backbone:

- Vision: ImageNet, COCO, segmentation (Mask R-CNN), detection (YOLO v3).
- NLP: residuals inside Transformers (attention + MLP).
- Audio, graphs, and reinforcement learning—everywhere residuals ease optimization.

1.6 Chapter Overview

This chapter explores the complete picture of residual learning:

- **Part 1:** Skip Connections — motivation, mathematics, and intuition.
- **Part 2:** Residual Blocks — basic, bottleneck, and pre-activation forms with TikZ diagrams.
- **Part 3:** CNN vs ResNet — structural and gradient-flow comparisons.
- **Part 4:** Theoretical Insights — ODE interpretation, training, and modern hybrids.

Together, these sections form a self-contained, mathematical, and visual understanding of why ResNets remain a cornerstone of deep learning.

Chapter 2

Skip Connections and Residual Learning

2.1 Motivation

Deep networks promise richer representations but suffer from *degradation*—adding layers beyond a certain depth increases training error rather than reducing it. This is not overfitting: even the training loss worsens, revealing an optimization bottleneck.

Let $\mathcal{H}(x)$ be the target mapping a few stacked layers should learn. Instead of learning $\mathcal{H}(x)$ directly, ResNets propose to learn the residual:

$$\mathcal{F}(x) = \mathcal{H}(x) - x \quad \Rightarrow \quad \mathcal{H}(x) = \mathcal{F}(x) + x.$$

If the identity mapping is optimal, then $\mathcal{F}(x) \approx 0$; learning such a perturbation is far easier than relearning $\mathcal{H}(x)$ from scratch. This re-parameterization transforms a deep nonlinear system into one that behaves like a perturbation of the identity.

2.2 What Are Skip Connections?

A **skip connection** (or **shortcut**) is a direct link that bypasses one or more intermediate layers, carrying either the original input or a linearly transformed version of it to a later layer.

- It provides an unimpeded path for both *information flow* (forward pass) and *gradient flow* (backward pass).
- It enables the network to represent the identity function easily, preventing degradation with depth.
- It encourages *feature reuse*: higher layers can access low-level cues directly.

Mathematically, for input x and transformation $\mathcal{F}(x)$,

$$y = \mathcal{F}(x) + x.$$

The addition operation fuses two feature maps of the same shape; if shapes differ, a projection $W_s x$ is used instead.

2.3 Why Skip Connections Work

The backward derivative for loss $\mathcal{L}$ through the block is

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y}\left(I + \frac{\partial \mathcal{F}}{\partial x}\right).$$

Even if $\frac{\partial \mathcal{F}}{\partial x}$ is small, the identity matrix I ensures gradient flow never vanishes. This simple algebraic property explains why ResNets scale to hundreds of layers.

2.4 Skip Connection Concept

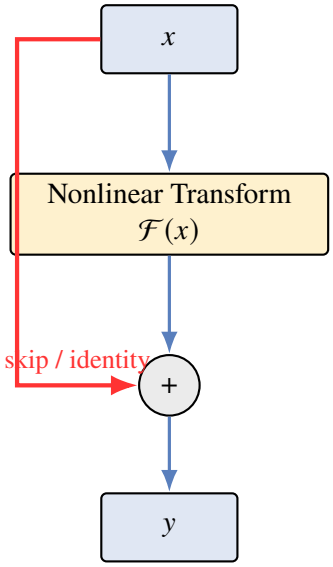
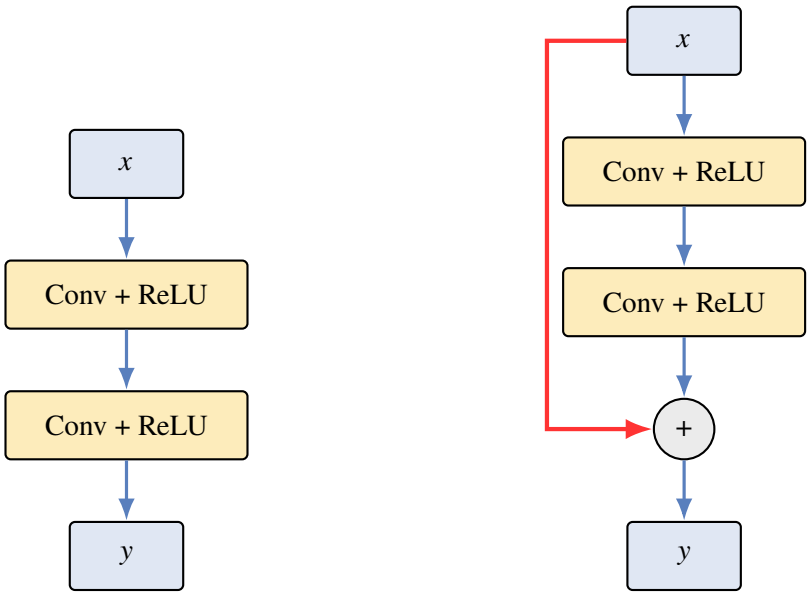


Figure 2.1: Skip connection: direct identity path added to a nonlinear transformation.

2.5 Comparison to Plain CNN Block

In a plain CNN, each layer must reconstruct both new and already learned information. If the optimal mapping is close to identity, this is inefficient. Skip connections relieve this burden by letting the network simply adjust residuals.



(a) Plain CNN block (b) With skip connection

Figure 2.2: Adding the identity path transforms the same convolutional pair into a residual block that trains deeper and faster.

2.6 Effect on Optimization Landscape

Skip connections smooth the effective loss surface. They create near-linear paths between shallow and deep solutions, allowing SGD to traverse valleys more directly.

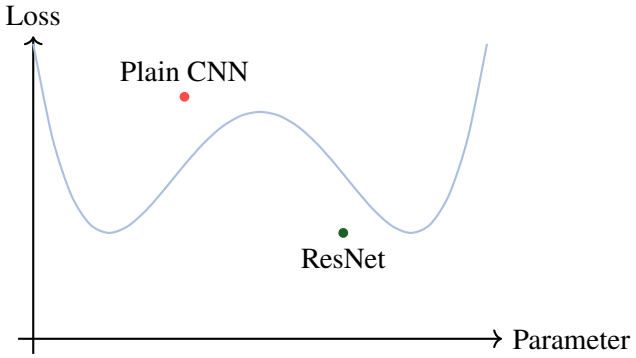


Figure 2.3: Residual parameterization leads to smoother, well-conditioned minima.

2.7 Summary

- Skip connections add the input directly to the output of a few stacked layers.
- They mitigate vanishing gradients, simplify identity mappings, and enable deeper models.
- In practice, every modern deep network—ResNets, Transformers, UNets—uses some form of skip/shortcut connection.

Chapter 3

Residual Blocks: Architecture and Intuition

3.1 What is a Residual Block?

A **residual block** is the fundamental building unit of a ResNet. Instead of passing information strictly sequentially, the block explicitly adds the input back to its own transformed output. This yields a local equation:

$$y = \mathcal{F}(x, W) + x,$$

where $\mathcal{F}(x, W)$ denotes the residual mapping (typically 2–3 convolutions). Conceptually, a residual block behaves like a "refinement unit" that nudges features rather than rewriting them.

- $\mathcal{F}(x, W)$ is a learnable nonlinear residual correction.
- x is the shortcut input (identity or projected).
- The summation merges both pathways to form y.

Such design ensures that if deeper layers are unhelpful, the block can learn $\mathcal{F}(x) = 0$ and simply copy the identity, keeping optimization stable.

3.2 Basic Residual Block (ResNet-18/34)

Structure

- Two 3×3 convolutional layers, each followed by BatchNorm and ReLU.
- The input is directly added to the output after the second BatchNorm.
- A final ReLU activates the sum.

Mathematics

$$\begin{aligned} z_1 &= \mathrm{ReLU}(\mathrm{BN}(W_1 * x)), \\ z_2 &= \mathrm{BN}(W_2 * z_1), \\ y &= \mathrm{ReLU}(z_2 + x). \end{aligned}$$

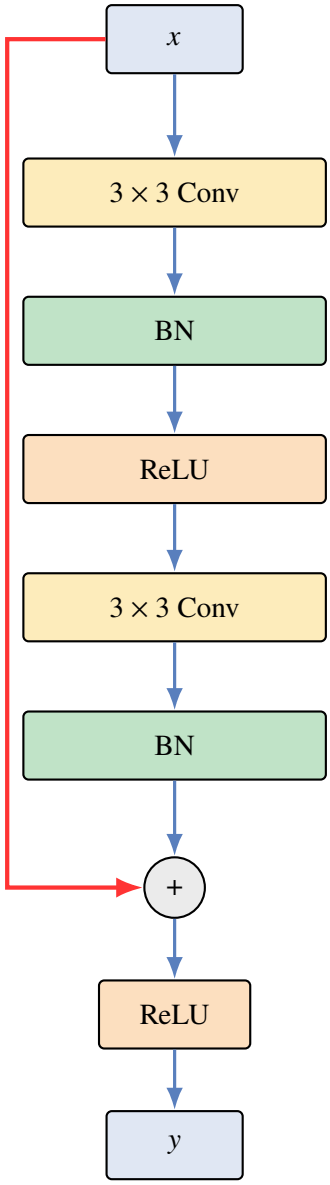


Figure 3.1: Basic residual block: two convolutions with identity shortcut.

Basic Block

Discussion

The key innovation is the additive identity path. If the block learns nothing useful, $\mathcal{F}(x) = 0$ and $y = x$. This makes stacking dozens of such blocks feasible—errors and gradients can propagate without distortion.

3.3 Bottleneck Residual Block (ResNet-50/101/152)

Motivation

As networks grow deeper, direct 3×3 stacks become expensive. The **bottleneck block** introduces a parameter-efficient alternative using three convolutions:

$$1 \times 1 \text{ (reduce)} \;\rightarrow\; 3 \times 3 \text{ (process)} \;\rightarrow\; 1 \times 1 \text{ (expand)}.$$

The first 1×1 reduces channel dimensionality, the middle 3×3 performs spatial mixing, and the last 1×1 restores the original width.

Mathematics

$$y = x + W_3 * \sigma(\mathrm{BN}(W_2 * \sigma(\mathrm{BN}(W_1 * x)))),$$

where σ denotes ReLU. This triple-conv scheme cuts FLOPs while retaining representational power.

Bottleneck Block

Advantages

- Reduces parameter count by up to 75% in deep stages.
- Improves GPU memory efficiency and batch size.
- Enables 50+ layer networks without prohibitive cost.

3.4 Pre-Activation Residual Block

Motivation

He et al. (2016) proposed swapping the order of BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv so that the shortcut path remains a *pure identity*—activations do not alter it. This **pre-activation** form improved regularization and made even 1000-layer networks trainable.

Formulation

$$y = x + W_2 * \sigma(\mathrm{BN}(W_1 * \sigma(\mathrm{BN}(x)))).$$

Pre-Activation Block

Impact

- Gradients pass entirely unchanged along the skip path.
- The residual branch learns a clean perturbation.
- Empirically improves convergence and test accuracy.

3.5 How Residual Blocks Differ from Plain CNN Layers

3.6 Connecting to Deeper Architectures

Stacking many residual blocks produces stage-wise architectures such as ResNet-18, 34, 50, etc. Each stage doubles the channel dimension while halving the spatial size. The next chapter visualizes these full pipelines and contrasts them with traditional CNNs.

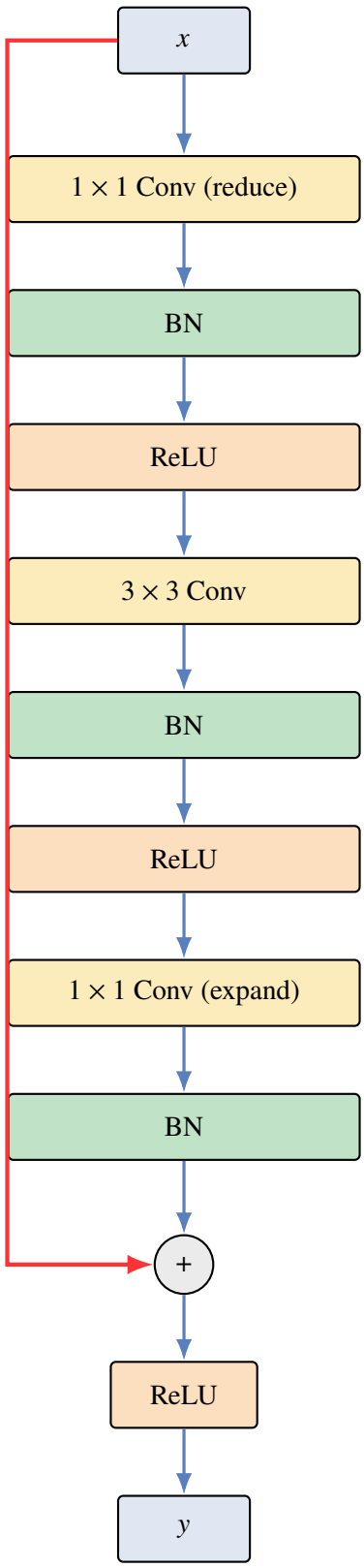


Figure 3.2: Bottleneck block: efficient triple-conv design used in deep ResNets.

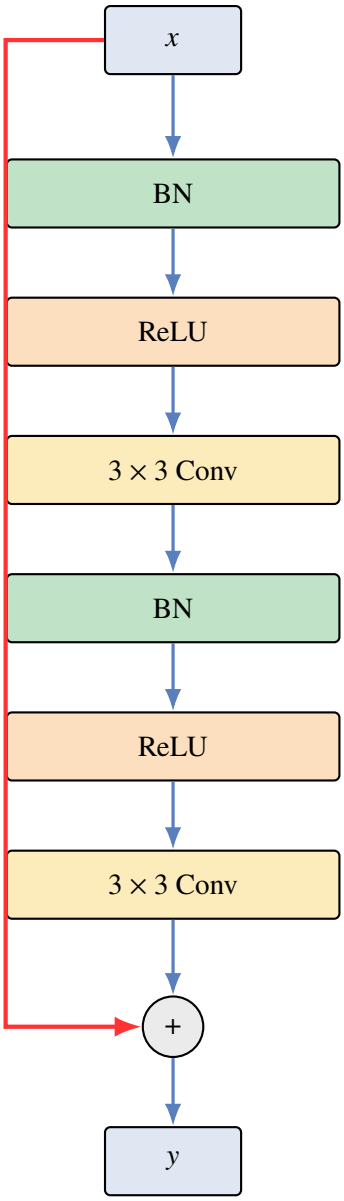


Figure 3.3: Pre-activation residual block: activation + normalization come before convolutions.

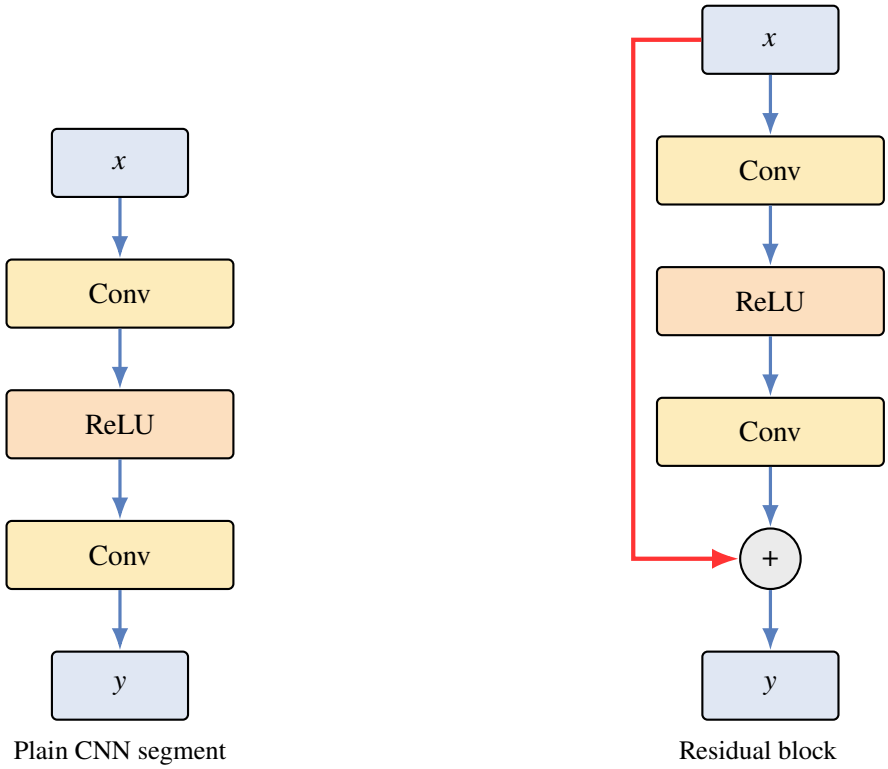


Figure 3.4: Residual blocks explicitly preserve identity information, plain CNNs do not.

Chapter 4

Traditional CNN vs ResNet: Architectural Comparison

4.1 Conceptual Differences

- **Traditional CNNs** process information sequentially. Each layer receives the transformed output of the previous one. As depth increases, signals (and gradients) may vanish or explode.
- **ResNets** add identity shortcuts across stacked layers, enabling stable deep training. Gradients can flow directly through identity paths, preventing attenuation.

4.2 Key Structural Contrast

In a traditional CNN, the mapping is purely compositional:

$$x_{l+1} = f_l(x_l),$$

where f_l denotes convolution, normalization, and activation.

In a ResNet, each block learns only the residual:

$$x_{l+1} = x_l + \mathcal{F}_l(x_l),$$

making the entire network effectively a series of incremental refinements.

4.3 Side-by-Side Architecture Visualization

4.4 Gradient Flow Comparison

In a plain CNN:

$$\frac{\partial \mathcal{L}}{\partial x_1} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=1}^{L-1} \frac{\partial x_{l+1}}{\partial x_l}.$$

Each Jacobian term can shrink the gradient, leading to vanishing signals.

In a ResNet:

$$\frac{\partial \mathcal{L}}{\partial x_1} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=1}^{L-1} \left(I + \frac{\partial \mathcal{F}_l}{\partial x_l} \right),$$

so the product always contains an identity component, maintaining stable magnitude.

4.5 Empirical Behavior

Empirical studies show that, on datasets like CIFAR-10 and ImageNet:

- Plain 56-layer CNNs have higher training error than 20-layer models (degradation).
- ResNet-56 achieves lower training and test error than both, thanks to skip connections.
- The improvement is not due to regularization but better optimization conditioning.

4.6 Optimization Landscape Intuition

Residual learning flattens the effective loss surface, reducing saddle points and allowing SGD to converge to broad minima.

4.7 Summary of Differences

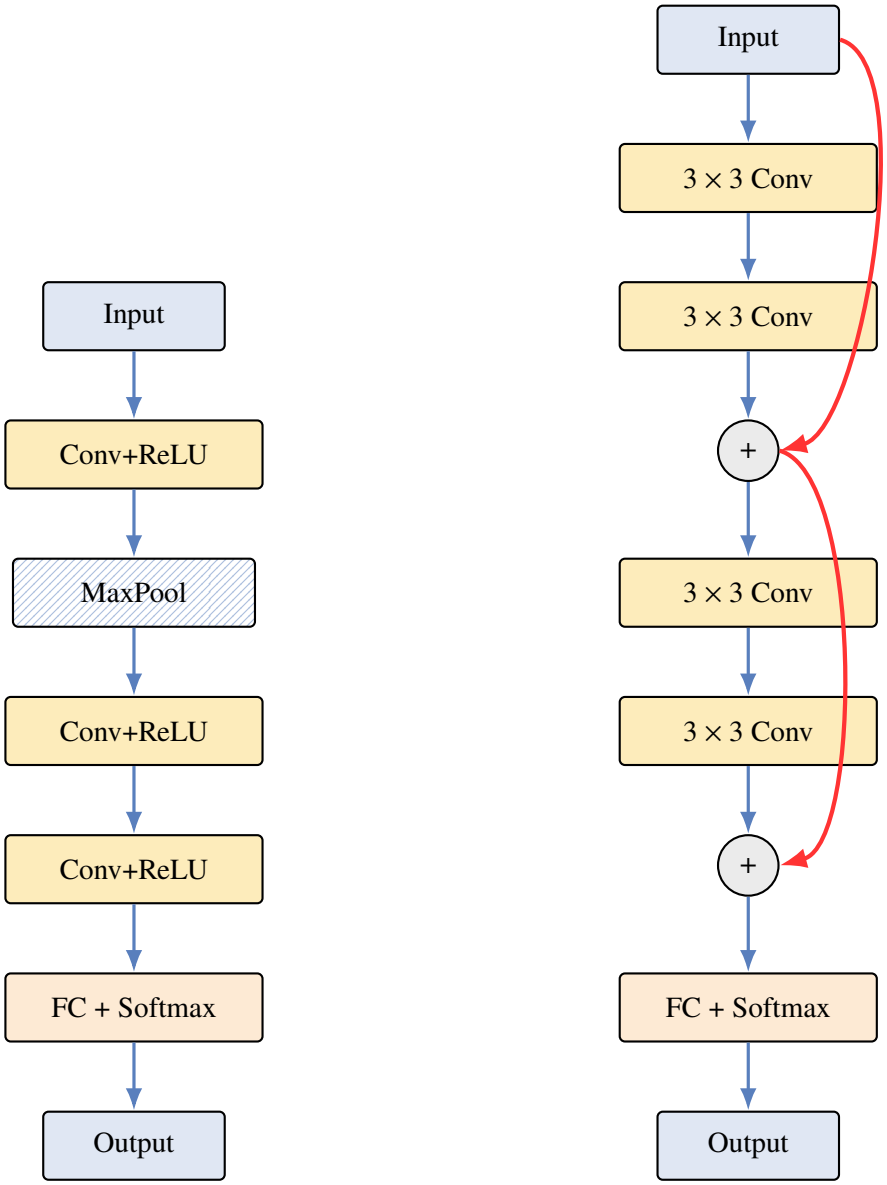
Aspect	**Plain CNN**	**ResNet**
Information flow	Strictly sequential	Parallel (main + skip path)
Optimization	Vanishing gradients possible	Stable via identity path
Representation	Must relearn all features	Learns residual corrections
Depth scalability	Limited (<30 layers)	Hundreds or more
Parameter efficiency	Lower	Similar but deeper due to stability
Training behavior	May degrade with depth	Improves with depth

Table 4.1: Comparison between plain CNNs and ResNets.

4.8 Practical Perspective

From a practitioner's viewpoint:

- Residual networks behave like ensembles of shallower sub-paths.
- During fine-tuning, early convolutional layers remain stable thanks to identity preservation.
- Almost every modern architecture—DenseNet, ConvNeXt, Vision Transformer—retains some residual form.



(a) Plain CNN pipeline (b) ResNet pipeline (two residual blocks)

Figure 4.1: Traditional CNN vs ResNet. Residuals introduce identity paths across pairs of convolutions, improving gradient flow.

Figure 4.2: Gradient flow in plain CNN vs ResNet block. Residual identity creates a direct gradient path.

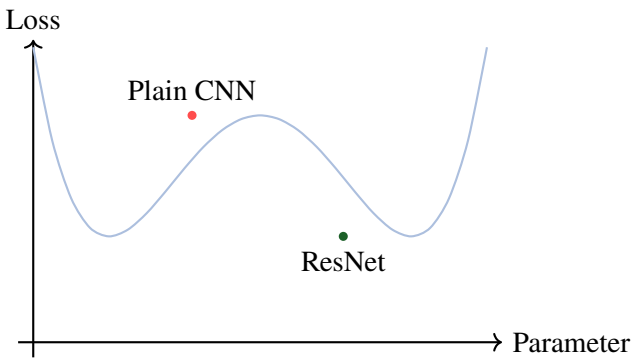


Figure 4.3: ResNet's residual parameterization smooths the optimization landscape.

Chapter 5

Theoretical Insights and Practical Implementation

5.1 Residual Networks as Discretized ODEs

A striking way to interpret ResNets is through the lens of **continuous-time dynamics**. Each residual block performs an update:

$$x_{l+1} = x_l + \mathcal{F}_l(x_l).$$

If we view l as discrete time steps with step size $\Delta t = 1$, then

$$\frac{x_{l+1} - x_l}{\Delta t} = \mathcal{F}_l(x_l) \quad \Rightarrow \quad \frac{dx}{dt} = f(x(t)),$$

which is the standard form of an ordinary differential equation (ODE). Thus, ResNets can be regarded as the *forward-Euler discretization* of a continuous flow of features through depth.

Implications

- The network depth corresponds to integration time.
- Continuous formulations inspire **Neural ODEs**, where backpropagation is done via adjoint sensitivity.
- Residual pathways naturally preserve information, like stable numerical integrators.

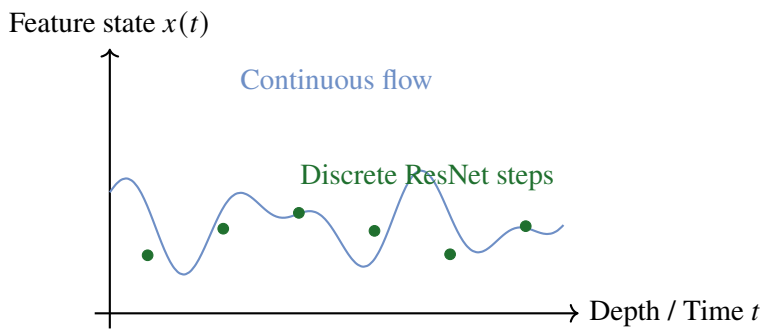


Figure 5.1: ResNet updates as Euler discretization of a continuous ODE.

5.2 Gradient Flow and Stability

Consider the recursive update:

$$x_{l+1} = x_l + \mathcal{F}(x_l), \quad \text{then} \quad \frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_{l+1}} \left(I + \frac{\partial \mathcal{F}}{\partial x_l} \right).$$

Products of $(I + \cdot)$ stay close to the identity, preserving gradient norm. This underlies the empirical stability of very deep residual models.

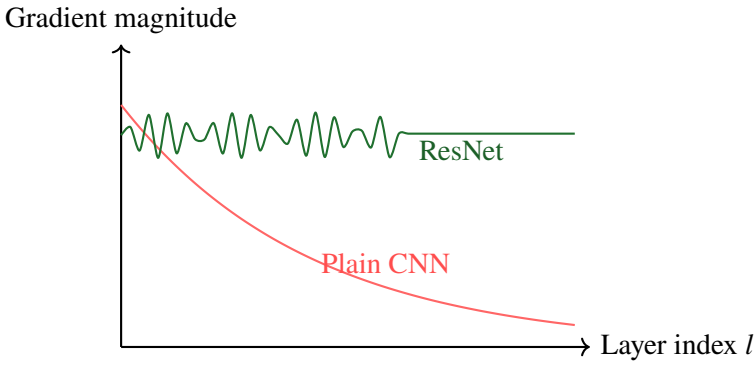


Figure 5.2: Gradient norms across layers: stable propagation in ResNets.

5.3 Training and Implementation Notes

Initialization

Residuals benefit from near-identity initialization. Setting the final BatchNorm scale γ to zero ensures the block initially performs an identity mapping, allowing deeper convergence.

Normalization

BatchNorm reduces internal covariate shift and is used after every convolution. For small batches, GroupNorm or LayerNorm are viable alternatives.

Optimization

- **Optimizer:** SGD with momentum (0.9) or AdamW.
- **Learning rate:** cosine decay or step schedule.
- **Regularization:** weight decay (10^{-4}), label smoothing (0.1).
- **Tricks:** stochastic depth, mixup, CutMix, AutoAugment.

Practical Considerations

- For 224×224 inputs, use 7×7 stem conv + max-pool.
- Each subsequent stage halves spatial resolution and doubles channels.
- Global average pooling precedes the final FC classifier.

5.4 From ResNets to Modern Hybrids

Residual ideas pervade nearly all modern architectures:

- **Wide ResNet** — increases channel width to boost capacity.
- **ResNeXt** — aggregates grouped convolutions (cardinality).
- **SE-ResNet** — adds channel-wise attention (Squeeze–Excite).
- **DenseNet** — connects each layer to all previous outputs.
- **ConvNeXt** — modernizes ResNets with large-kernel convs and LayerNorm.
- **Vision Transformers** — replace local convs with global self-attention but retain residual blocks.

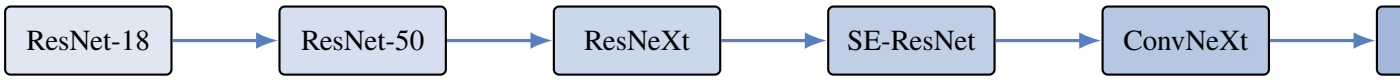


Figure 5.3: Evolution of architectures derived from ResNet principles.

5.5 Applications and Impact

- **Vision:** classification, detection (Faster-R-CNN, YOLOv3), segmentation (Mask R-CNN, U-Net).
- **Medical AI:** diabetic-retinopathy grading, chest-X-ray diagnosis, MRI segmentation.
- **Audio:** spectrogram classification and speech recognition.
- **Graphs:** Graph ResNets enhance node classification.
- **Reinforcement Learning:** ResNet backbones stabilize visual policies.

5.6 Limitations and Open Directions

- Computational cost increases linearly with depth.
- Memory overhead remains significant for dense residuals.
- Skip connections assume identical tensor shapes; projections add parameters.
- Scaling beyond 1000 layers yields diminishing returns.
- Research continues into dynamic residual paths and adaptive pruning.

References

1. K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016.
2. K. He et al., *Identity Mappings in Deep Residual Networks*, ECCV 2016.
3. S. Ioffe, C. Szegedy, *Batch Normalization*, ICML 2015.
4. K. Simonyan, A. Zisserman, *Very Deep Convolutional Networks*, arXiv:1409.1556.
5. S. Zagoruyko, N. Komodakis, *Wide Residual Networks*, BMVC 2016.

6. S. Xie et al., *Aggregated Residual Transformations for Deep Neural Networks*, CVPR 2017.
7. J. Hu, L. Shen, G. Sun, *Squeeze-and-Excitation Networks*, CVPR 2018.
8. G. Huang et al., *Densely Connected Convolutional Networks*, CVPR 2017.
9. R. T. Chen et al., *Neural Ordinary Differential Equations*, NeurIPS 2018.
10. T. He et al., *Bag of Tricks for Image Classification*, CVPR 2019.
11. K. Liu et al., *ConvNeXt: A ConvNet for the 2020s*, CVPR 2022.
12. A. Dosovitskiy et al., *An Image is Worth 16×16 Words*, ICLR 2021.